

Volume 5 – Agent Runtime Reference Architecture

Version: 1.0

Purpose

This document defines the runtime architecture for the Enterprise Tiny Agent Framework.

Previous volumes defined:

- Agent Principles
- Engineering Practices
- Agent SDK
- Tool SDK

This document defines:

How Everything Runs Together

The runtime architecture provides:

- Agent execution
- Tool execution
- Model routing
- Memory management
- Security
- Observability
- Auditing
- Event processing
- Configuration management

This architecture is optimized for:

Java 21+
Spring Boot
DuckDB
Calcite
PostgreSQL
Local LLMs
Hosted LLMs

Runtime Philosophy

The runtime should be:

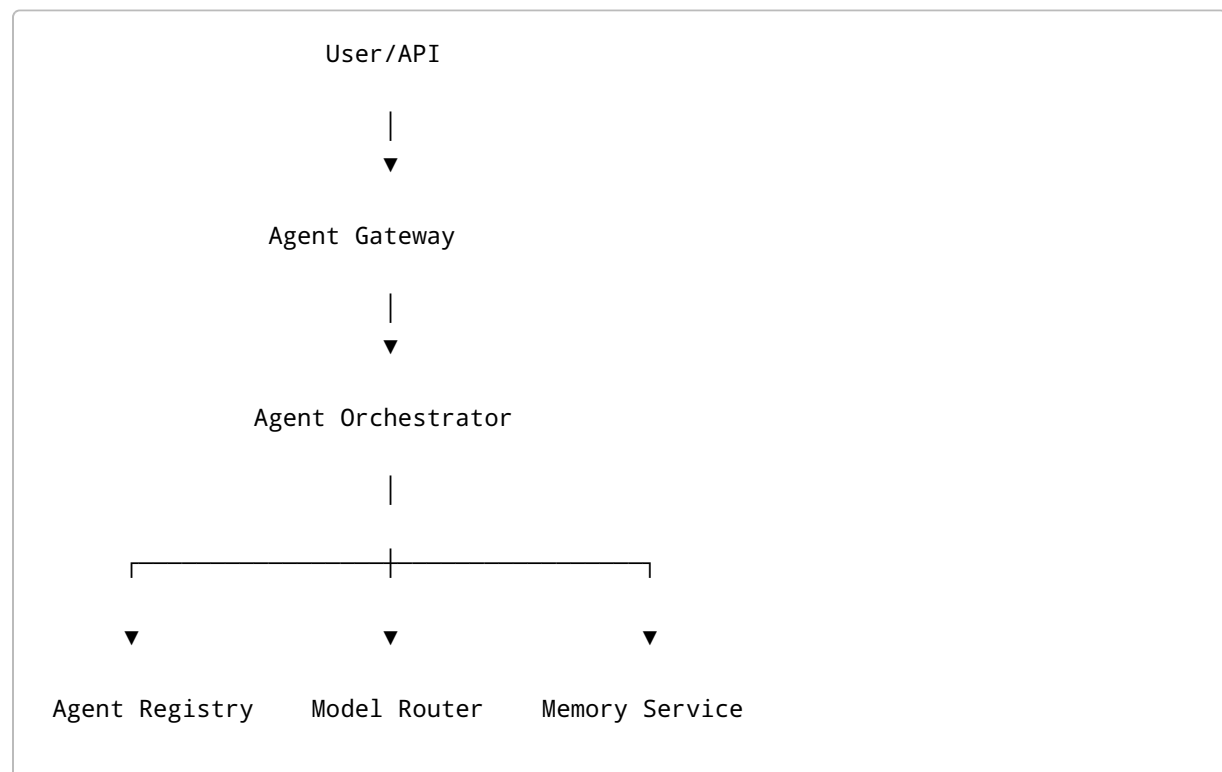
Simple
Observable
Deterministic
Composable
Replaceable

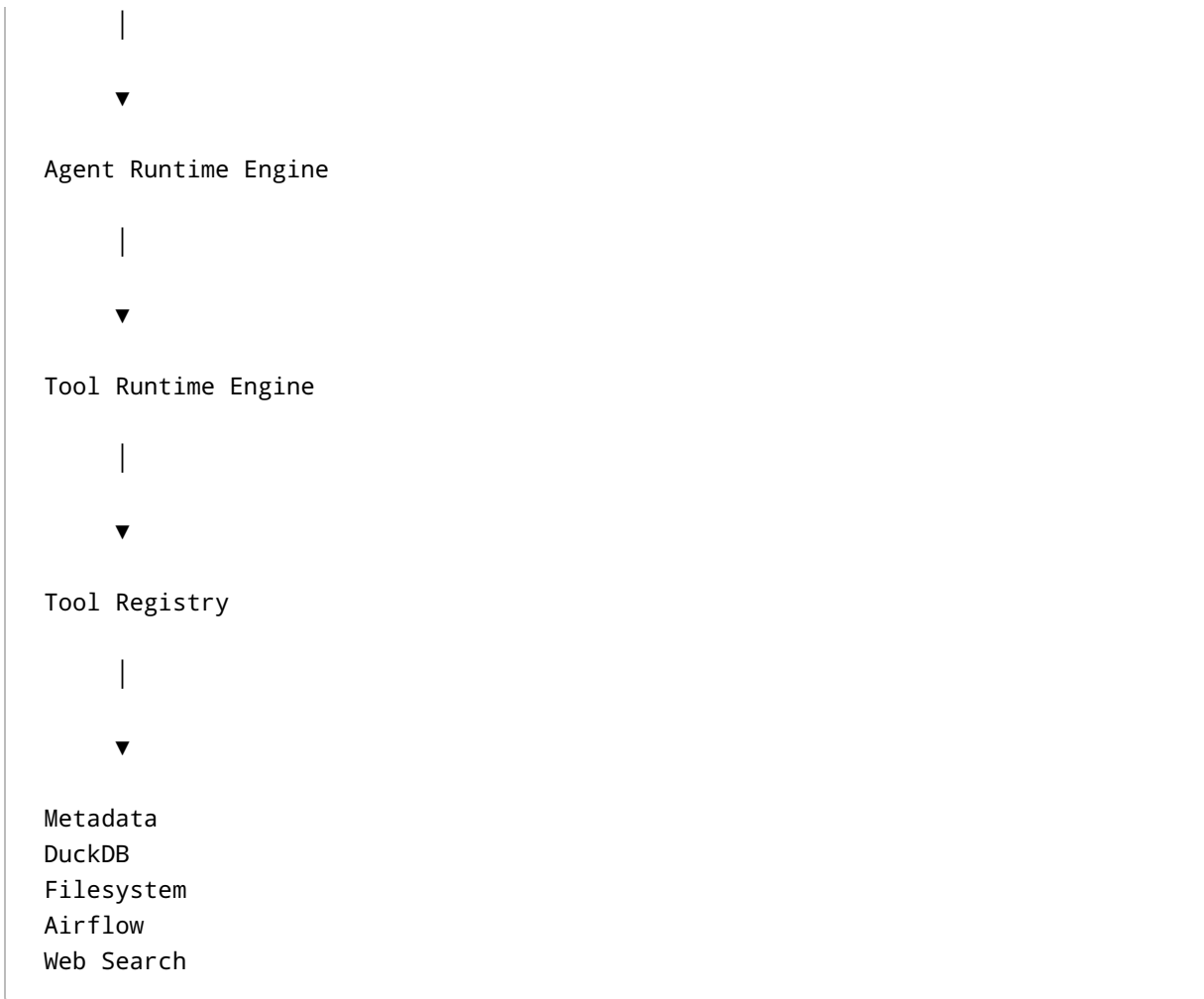
Avoid:

Magic
Reflection-heavy systems
Autonomous swarms
Hidden execution paths

Everything should be traceable.

High-Level Architecture





Runtime Modules

Recommended Maven structure:

```
agent-platform
├─ agent-api
├─ agent-runtime
├─ agent-orchestrator
├─ tool-api
├─ tool-runtime
```

- └─ model-api
- └─ model-router
- └─ memory-api
- └─ memory-runtime
- └─ audit-runtime
- └─ event-runtime
- └─ security-runtime
- └─ agent-starter
- └─ agent-server

Agent Gateway

Purpose:

Single entry point.

Examples:

REST API

WebSocket

CLI

MCP

Chat Interface

Responsibilities

Authentication

Authorization

Request Tracking

Request Validation

Correlation ID Creation

Example

`POST /agents/execute`

Request:

```
{
  "agent": "metadata-agent",
  "input": "Find subscriber datasets"
}
```

Agent Orchestrator

Purpose:

Coordinate execution.

The orchestrator does NOT execute business logic.

Responsibilities

Agent Selection

Request Routing

Execution Monitoring

Failure Handling

Response Aggregation

Interface

```
public interface AgentOrchestrator {  
  
    AgentResponse process(  
        AgentRequest request);  
  
}
```

Agent Registry

Purpose:

Maintain all registered agents.

Interface

```
public interface AgentRegistry {  
  
    Agent getAgent(  
        String agentId);  
  
}
```

Startup

Spring scans:

```
@Component
public class SqlAgent
```

Registration:

```
AgentRegistry.register(
    sqlAgent);
```

Agent Runtime Engine

Purpose:

Execute agent lifecycle.

Lifecycle

```
Initialize
  ↓
Validate
  ↓
Build Context
  ↓
Plan
  ↓
Execute
  ↓
Generate Response
  ↓
Audit
```

Interface

```
public interface AgentRuntime {

    AgentResponse execute(
        Agent agent,
        AgentRequest request);
```

```
}
```

Agent Execution Context

Purpose:

Hold all execution state.

Structure:

```
public class AgentExecutionContext {  
  
    private String requestId;  
  
    private String sessionId;  
  
    private String agentId;  
  
    private String userInput;  
  
    private AgentMemory memory;  
  
    private Map<String, Object> attributes;  
  
    private List<ToolResult> toolResults;  
  
}
```

Planning Engine

Purpose:

Convert request into executable actions.

Output:

AgentPlan

Example:

Search Dataset
Validate Schema
Generate SQL

Tool Runtime Engine

Purpose:

Execute tools consistently.

Responsibilities

Validation
Authorization
Execution
Metrics
Auditing

Interface

```
public interface ToolRuntime {  
  
    ToolResult execute(  
        Tool tool,  
        ToolRequest request);  
  
}
```

Tool Registry

Purpose:

Store all available tools.

Interface

```
public interface ToolRegistry {  
  
    Tool getTool(  
        String toolId);  
  
}
```

Discovery

Spring Boot:

```
applicationContext  
    .getBeansOfType(Tool.class)
```

Model Layer

Purpose:

Abstract all LLM providers.

Model API

```
public interface ModelProvider {  
  
    LlmResponse generate(  
        LlmRequest request);  
  
}
```

```
}
```

Implementations

```
OllamaProvider  
  
OpenAIProvider  
  
LMStudioProvider  
  
vLLMProvider  
  
ClaudeProvider
```

Model Router

Purpose:

Select model dynamically.

Responsibilities

```
Cost Optimization  
  
Capability Matching  
  
Fallback Routing  
  
Escalation
```

Example:

```
Metadata Agent  
  → Qwen 4B
```

Research Agent

→ Qwen 14B

Escalation

→ Hosted Model

Memory Runtime

Purpose:

Manage conversation state.

Memory Types

Request Memory

Exists only during execution.

Session Memory

Exists during conversation.

Persistent Memory

Optional.

Use carefully.

Interface

```
public interface MemoryProvider {  
  
    AgentMemory load(  
        String sessionId);  
  
    void save(  
        AgentMemory memory);  
}
```

```
}
```

Event Runtime

Purpose:

Publish execution events.

Event Types

```
AGENT_STARTED  
  
AGENT_COMPLETED  
  
AGENT_FAILED  
  
TOOL_STARTED  
  
TOOL_COMPLETED  
  
TOOL_FAILED  
  
MODEL_CALLED  
  
MODEL_FAILED
```

Event Interface

```
public interface EventPublisher {  
  
    void publish(  
        RuntimeEvent event);  
  
}
```

Event Storage

Recommended:

PostgreSQL

Optional:

Kafka

for high volume systems.

Audit Runtime

Purpose:

Compliance and troubleshooting.

Audit Record

```
public class AuditEvent {  
  
    private String eventId;  
  
    private String requestId;  
  
    private String agentId;  
  
    private String action;  
  
    private Instant timestamp;  
  
}
```

Security Runtime

Purpose:

Enforce permissions.

Security Layers

Gateway Security

Authentication

Agent Security

Allowed Tools

Tool Security

Operation Permissions

Example

```
Metadata Agent
```

```
Allowed:
```

```
    SearchDatasetTool
```

```
Forbidden:
```

```
    RestartJobTool
```

Configuration Runtime

Purpose:

Externalized configuration.

Example

```
agents:
```

```
sql-agent:

  preferred-model: qwen-4b

  fallback-model: qwen-14b

tools:

  metadata-search:

    cache-enabled: true

models:

  qwen-4b:

    provider: ollama
```

Caching Layer

Purpose:

Reduce expensive operations.

Recommended:

```
Metadata Lookup

Schema Lookup

Documentation Retrieval
```

Avoid:

```
Job Status

Live Monitoring
```

Error Handling

Standard error model.

Error Categories

```
ValidationError  
  
AuthorizationError  
  
ToolExecutionError  
  
ModelError  
  
SystemError
```

Response Example

```
{  
  "success": false,  
  "errorCode": "TOOL_EXECUTION_ERROR",  
  "message": "Metadata search failed"  
}
```

Agent Escalation Engine

Purpose:

Handle low confidence outcomes.

Strategies:

```
Retry  
  
Alternative Tool
```

Larger Model

Human Review

Example:

Qwen 4B

Confidence 0.62

↓

Qwen 14B

Confidence 0.88

↓

Return Result

Deployment Topology

Development

Spring Boot

PostgreSQL

Ollama

Local Filesystem

Enterprise

Agent Server Cluster

PostgreSQL

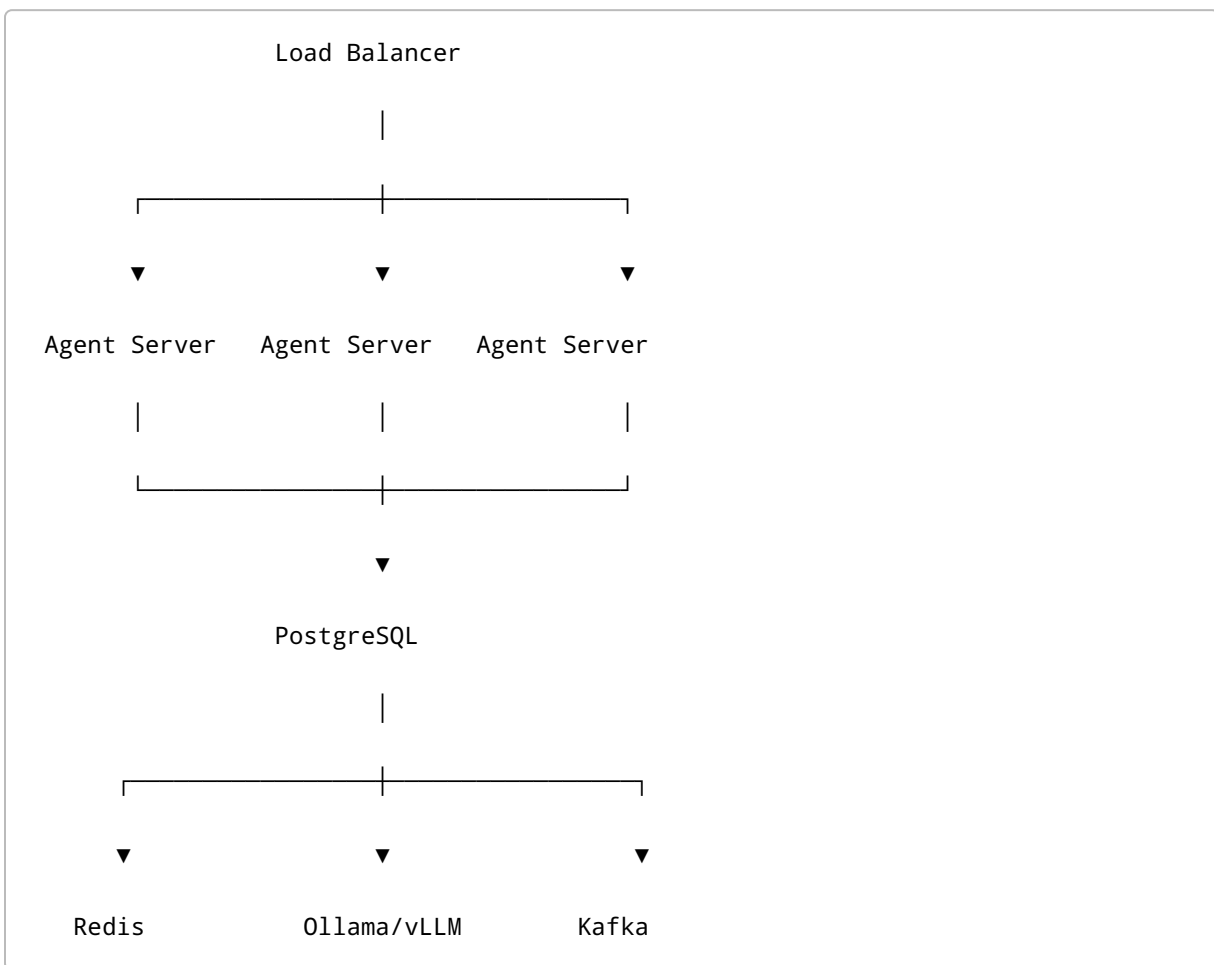
Redis

Kafka

vLLM

Hosted Models

Reference Deployment



Runtime Metrics

Every component should expose:

Requests

Failures

Execution Time

Tool Invocations

Token Usage

Model Cost

Cache Hit Rate

Health Checks

Every runtime service should expose:

`/health`

`/readiness`

`/liveness`

Runtime Testing Strategy

Unit Tests

Runtime Components

Integration Tests

Runtime + Agent + Tool

Load Tests

Concurrent Agent Requests

Concurrent Tool Execution

Model Throughput

MCP Integration Strategy

MCP is NOT part of the runtime core.

MCP should be an adapter.

Agent Runtime

↓

MCP Adapter

↓

Claude Desktop

Cursor

VS Code

External Clients

This keeps business logic independent of protocols.

Recommended Initial Build Order

Phase 1

Agent Registry

Tool Registry

Agent Runtime

Tool Runtime

Phase 2

Model Router

Memory Service

Audit Service

Phase 3

Event Runtime

Caching

Escalation Engine

Phase 4

MCP Adapter

Distributed Deployment

Advanced Routing

Final Architecture Rule

The runtime exists to make Agents and Tools boring.

Adding a new Agent should require:

Create Agent Class

Register Agent

Configure Tools

Deploy

Adding a new Tool should require:

Create Tool Class

Register Tool

Deploy

If adding a new capability requires modifying the runtime, the runtime has become too tightly coupled and should be refactored.